



MODUL PRAKTIKUM

Disusun Oleh:

Aprilia Kurnianti, S.T., M.Eng.
Dosen Pengampu Matkul IBD

Contents

Materi 10 DATABASE TESTING.....	3
A. Konsep Pengujian Database.....	3
PRAKTIKUM 1 — MENYUSUN TEST CASE DATABASE.....	3
PRAKTIKUM 2 — FUNCTIONAL TESTING.....	4
PRAKTIKUM 3 — FUNCTIONAL TESTING INSERT	4
PRAKTIKUM 4 — FUNCTIONAL TESTING UPDATE	5
PRAKTIKUM 5 — FUNCTIONAL TESTING DELETE.....	5
PRAKTIKUM 6 — INTEGRITY TESTING PRIMARY KEY	6
PRAKTIKUM 7 — INTEGRITY TESTING FOREIGN KEY	6
PRAKTIKUM 8 — INTEGRITY TESTING CHECK CONSTRAINT	7
PRAKTIKUM 9 — INTEGRITY TESTING NULL	7
PRAKTIKUM 10 — TRANSACTION TESTING: COMMIT	8
PRAKTIKUM 11 — TRANSACTION TESTING: ROLLBACK	9
PRAKTIKUM 12 — TRANSACTION TESTING DENGAN SAVEPOINT	10
PRAKTIKUM 13 — PERFORMANCE TESTING.....	10
PRAKTIKUM 14 — PERFORMANCE TESTING QUERY DENGAN FILTER	11
PRAKTIKUM 15 — SECURITY TESTING DASAR	12
PRAKTIKUM 16 — Membandingkan Query Aman dan Tidak Aman.....	14
PRAKTIKUM 17 — Parameterized Query	15

Materi 10

Pengujian Database

A. Konsep Pengujian Database

Pengujian database dilakukan untuk memastikan bahwa database tidak hanya dapat menyimpan data, tetapi juga dapat menjalankan fungsi sesuai kebutuhan, menjaga integritas data, menjalankan transaksi dengan benar, memiliki performa yang memadai, dan memiliki perlindungan dasar terhadap akses yang tidak semestinya. Pada praktikum ini pengujian dibagi menjadi lima jenis:

Jenis Testing	Fokus
Functional Testing	Apakah fungsi database berjalan sesuai kebutuhan?
Integrity Testing	Apakah integritas data tetap terjaga?
Transaction Testing	Apakah transaksi berhasil dan dapat dibatalkan dengan benar?
Performance Testing	Seberapa baik performa query/database?
Security Testing Dasar	Apakah akses dan input database memiliki perlindungan dasar?

Gunakan:

```
USE DB_Petshop;  
GO
```

Pastikan tabel tersedia:

```
SELECT TABLE_NAME  
FROM INFORMATION_SCHEMA.TABLES  
WHERE TABLE_TYPE = 'BASE TABLE';
```

Sebelum melakukan pengujian yang mengubah data, pastikan gunakan data uji dan tidak merusak data utama.

PRAKTIKUM 1 — MENYUSUN TEST CASE DATABASE

Sebelum menjalankan pengujian, buat daftar test case.

Format sederhana:

ID	Jenis Testing	Skenario	Input	Expected Result	Actual Result	Status
FT-01	Functional	Menampilkan produk	Query SELECT	Data tampil		
IT-01	Integrity	Produk tanpa kategori	Kode kategori salah	Ditolak		
TT-01	Transaction	Pembatalan transaksi	ROLLBACK	Data kembali		
PT-01	Performance	Query produk	SELECT	Waktu wajar		
ST-01	Security	Input tidak valid	' OR 1=1 --	Tidak dieksekusi sebagai SQL		

Status pengujian menggunakan:

- PASS
- FAIL

PRAKTIKUM 2 — FUNCTIONAL TESTING

1. Pengujian SELECT

Functional Testing digunakan untuk memastikan fungsi database berjalan sesuai kebutuhan.

Contoh:

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    Stok
FROM dbo.Produk;
```

Expected Result

Data produk dapat ditampilkan tanpa error.

Test Case	Expected	Actual	Status
FT-01	Data produk tampil		

2. Pengujian WHERE

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    Stok
FROM dbo.Produk
WHERE Stok > 10;
```

Expected Result

Hanya produk dengan stok lebih dari 10 yang ditampilkan.

Pertanyaan

1. Apakah semua data yang ditampilkan memenuhi kondisi?
2. Apakah ada data yang tidak sesuai?

PRAKTIKUM 3 — FUNCTIONAL TESTING INSERT

Pengujian berikut digunakan untuk memastikan database dapat menerima data baru.

Contoh:

```

INSERT INTO dbo.Produk
(
    KodeProduk,
    NamaProduk,
    Harga,
    Stok,
    KodeKategori
)
VALUES
(
    'PRD991',
    'Produk Testing',
    50000,
    10,
    'KTG001'
);

```

Kemudian periksa:

```

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD991';

```

Expected Result

Data PRD991 berhasil tersimpan.

Setelah pengujian selesai, data uji dapat dihapus:

```

DELETE FROM dbo.Produk
WHERE KodeProduk = 'PRD991';

```

PRAKTIKUM 4 — FUNCTIONAL TESTING UPDATE

Uji perubahan data:

```

UPDATE dbo.Produk
SET Harga = 55000
WHERE KodeProduk = 'PRD999';

```

70 %

Results Messages

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD999	Snack Kucing	KTG001	55000.00	30

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD999	Snack Kucing	KTG001	25000.00	30

Expected Result

Harga produk berubah menjadi 55000.

Test Case

ID	FT-02
Skenario	Update harga
Expected	Harga berubah
Actual	
Status	

PRAKTIKUM 5 — FUNCTIONAL TESTING DELETE

Uji penghapusan:

```
DELETE FROM dbo.Produk
WHERE KodeProduk = 'PRD999';
```

Kemudian:

```
SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD999';
```

Expected Result

Data PRD999 sudah tidak ditemukan.

PRAKTIKUM 6 — INTEGRITY TESTING PRIMARY KEY

Integrity Testing bertujuan memastikan aturan integritas database tetap bekerja. Coba masukkan kode produk yang sama dengan data yang sudah ada.

Misalnya:

```
INSERT INTO dbo.Produk
(
    KodeProduk,
    NamaProduk,
    Harga,
    Stok,
    KodeKategori
)
VALUES
(
    'PRD001',
    'Produk Duplikat',
    50000,
    10,
    'KTG001'
);
```

Expected Result

Database menolak data karena KodeProduk merupakan Primary Key dan tidak boleh duplikat.

Catat:

ID	IT-01
Pengujian	Duplicate Primary Key
Expected	INSERT ditolak
Actual	
Status	

PRAKTIKUM 7 — INTEGRITY TESTING FOREIGN KEY

Sekarang uji hubungan antara Produk dan KategoriProduk. Masukkan kode kategori yang tidak ada.

```

INSERT INTO dbo.Produk
(
    KodeProduk,
    NamaProduk,
    Harga,
    Stok,
    KodeKategori
)
VALUES
(
    'PRD998',
    'Produk Test FK',
    50000,
    10,
    'KTG999'
);

```

Expected Result

INSERT ditolak karena KTG999 tidak terdapat pada tabel kategori.

Kesimpulan

Foreign Key berfungsi untuk menjaga agar data anak tidak memiliki referensi ke data induk yang tidak tersedia.

PRAKTIKUM 8 — INTEGRITY TESTING CHECK CONSTRAINT

Apabila tabel memiliki CHECK CONSTRAINT, kita dapat menguji apakah aturan tersebut bekerja.

Misalnya terdapat aturan:

Harga > 0

Uji dengan:

```

INSERT INTO dbo.Produk
(
    KodeProduk,
    NamaProduk,
    Harga,
    Stok,
    KodeKategori
)
VALUES
(
    'PRD997',
    'Produk Test Check',
    -5000,
    10,
    'KTG001'
);

```

Expected Result

Data ditolak apabila terdapat CHECK CONSTRAINT yang melarang harga negatif.

Pertanyaan

1. Apakah constraint bekerja?
2. Apa pesan error yang muncul?
3. Apa manfaat constraint tersebut?

PRAKTIKUM 9 — INTEGRITY TESTING NULL

Uji kolom yang tidak boleh NULL.

Contoh:

```
INSERT INTO dbo.Produk
(
    KodeProduk,
    NamaProduk,
    Harga,
    Stok,
    KodeKategori
)
VALUES
(
    'PRD996',
    NULL,
    50000,
    10,
    'KTG001'
);
```

Expected Result

Jika NamaProduk memiliki constraint NOT NULL, database menolak data tersebut.

Kesimpulan

Pengujian ini memastikan database tidak menerima data kosong pada atribut yang wajib diisi.

PRAKTIKUM 10 — TRANSACTION TESTING: COMMIT

Transaction Testing bertujuan memastikan perubahan data dapat dikendalikan menggunakan transaksi.

Mulai transaksi:

```
BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD001';

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';

COMMIT TRANSACTION;
```

Expected Result

Perubahan stok disimpan secara permanen.

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	26

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	25

PRAKTIKUM 11 — TRANSACTION TESTING: ROLLBACK

Sekarang lakukan perubahan tetapi batalkan.

```
BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 5
WHERE KodeProduk = 'PRD001';

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';

ROLLBACK TRANSACTION;
```

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	25

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	20

Periksa kembali:

```
SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';
```

70 %

Results Messages

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	25

Expected Result

Perubahan stok dibatalkan.

Pertanyaan

1. Apa perbedaan hasil COMMIT dan ROLLBACK?
2. Mengapa ROLLBACK penting dalam transaksi?

PRAKTIKUM 12 — TRANSACTION TESTING DENGAN SAVEPOINT

Gunakan SAVE TRANSACTION.

```
UPDATE dbo.Produk
SET Stok = Stok - 2
WHERE KodeProduk = 'PRD001';

SAVE TRANSACTION TitikAman;

UPDATE dbo.Produk
SET Harga = Harga + 1000
WHERE KodeProduk = 'PRD001';

ROLLBACK TRANSACTION TitikAman;

COMMIT TRANSACTION;
```

70 %

Results Messages

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	23

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	25

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	76000.00	23

	KodeProduk	NamaProduk	KodeKategori	Harga	Stok
1	PRD001	Whiskas Tuna 1kg	KTG001	75000.00	23

Kemudian periksa:

```
SELECT
    KodeProduk,
    Harga,
    Stok
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';
```

Analisis

1. Perubahan stok dilakukan sebelum savepoint sehingga tetap disimpan.
2. Perubahan harga dilakukan setelah savepoint sehingga dibatalkan ketika:

ROLLBACK TRANSACTION TitikAman;

dijalankan.

PRAKTIKUM 13 — PERFORMANCE TESTING

Performance Testing dilakukan untuk mengetahui bagaimana database merespons sebuah query.

Gunakan query:

```
SELECT
    KodeProduk,
    NamaProduk,
    Harga,
    Stok
FROM dbo.Produk;
```

Aktifkan:

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
```

Jalankan query.

```

SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 1 ms.

(17 rows affected)
Table 'Produk'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob
SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 1 ms.

Completion time: 2026-08-23T13:40:59.1907767+07:00

```

Kemudian:

```

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

```

```

SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-08-23T13:41:50.8280011+07:00

```

Catat:

Parameter	Hasil
Logical Reads	
CPU Time	
Elapsed Time	

PRAKTIKUM 14 — PERFORMANCE TESTING QUERY DENGAN FILTER

Bandingkan dengan query yang menggunakan filter:

```

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT
  KodeProduk,
  NamaProduk,
  Harga,
  Stok
FROM dbo.Produk
WHERE Harga > 50000;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

```

70 %

Messages

```

SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-08-23T13:45:25.0925522+07:00

```

Bandingkan dengan Praktikum 13.

Parameter	SELECT Semua	SELECT dengan WHERE
Logical Reads		
CPU Time		
Elapsed Time		

Pertanyaan

1. Query mana yang membutuhkan lebih banyak resource?
2. Mengapa hasilnya berbeda?
3. Apakah hasil tersebut selalu sama apabila jumlah data berubah?

PRAKTIKUM 15 — SECURITY TESTING DASAR

Security Testing pada praktikum ini berfokus pada pemahaman dasar tentang keamanan query dan input pengguna. Mahasiswa mencoba memahami risiko apabila input pengguna langsung digabungkan ke dalam SQL.

Langkah 1 — Input Normal

Di SSMS, buat contoh query:

```
DECLARE @NamaPelanggan VARCHAR(100);  
SET @NamaPelanggan = 'Andi Saputra';  
  
SELECT *  
FROM Pelanggan  
WHERE NamaPelanggan = @NamaPelanggan;
```

Hasil yang diharapkan, Jika Andi Saputra ada di database:

Data Andi Saputra ditampilkan.

Jika tidak ada:

Tidak ada data yang ditampilkan.

Catat:

Pengujian	Input	Expected Result	Actual Result	Status
ST-01	Andi Saputra	Data Andi Saputra ditampilkan		

Langkah 2 — Input Tidak Ditemukan

Sekarang ubah:

```
SET @NamaPelanggan = 'Andi Saputra';
```

Query:

```
SELECT *  
FROM Pelanggan  
WHERE NamaPelanggan = @NamaPelanggan;
```

Expected Result

Tidak ada data yang ditampilkan.

Catat:

Pengujian	Input	Expected Result	Actual Result	Status
ST-02	PelangganTidakAda	Tidak ada data		

Langkah 3 - Input Kosong

Sekarang :

```
SET @NamaPelanggan = '';
```

Jalankan:

```
SELECT *  
FROM Pelanggan  
WHERE NamaPelanggan = @NamaPelanggan;
```

Pertanyaan

Apa yang terjadi?

Apakah:

1. query menghasilkan data?
2. query tidak menghasilkan data?
3. database menghasilkan error?

Catat hasil sebenarnya.

Langkah 4 - Input Karakter Khusus

Sekarang gunakan:

```
SET @NamaPelanggan = ''''';
```

Kemudian:

```
SELECT *  
FROM Pelanggan  
WHERE NamaPelanggan = @NamaPelanggan;
```

Yang harus diperhatikan

Apakah query:

1. tetap berjalan?
2. menghasilkan error?
3. menghasilkan data?

Tujuannya bukan melakukan serangan, tetapi menguji apakah input dengan karakter khusus dapat ditangani.

Langkah 5 - Input SQL Injection

Sekarang gunakan input pengujian:

```
' OR '1'='1
```

Masukkan sebagai nilai:

```
DECLARE @NamaPelanggan VARCHAR(100);  
SET @NamaPelanggan = '' OR ''1''=''1'';  
SELECT *  
FROM Pelanggan  
WHERE NamaPelanggan = @NamaPelanggan;
```

Hasil yang diharapkan

Query tidak menampilkan seluruh data pelanggan.

Mengapa?

Karena:

```
WHERE NamaPelanggan = @NamaPelanggan
```

menganggap:

```
' OR '1'='1
```

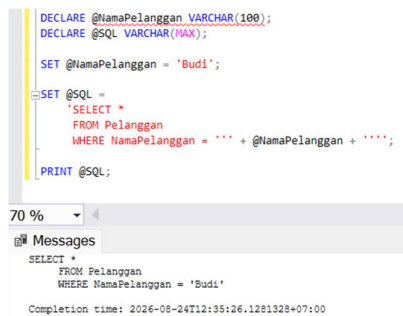
sebagai **nilai**, bukan sebagai bagian dari sintaks SQL.

PRAKTIKUM 16 — Membandingkan Query Aman dan Tidak Aman

Nah, bagian ini kita akan benar-benar melihat perbedaannya.

A. Query yang tidak aman

Simulasikan pembentukan query menggunakan string.



```
DECLARE @NamaPelanggan VARCHAR(100);
DECLARE @SQL VARCHAR(MAX);

SET @NamaPelanggan = 'Budi';

SET @SQL =
'SELECT *
FROM Pelanggan
WHERE NamaPelanggan = ''' + @NamaPelanggan + '''';

PRINT @SQL;
```

70 %

Messages

```
SELECT *
FROM Pelanggan
WHERE NamaPelanggan = 'Budi'
```

Completion time: 2026-09-24T12:35:26.1281328+07:00

Jalankan. Perhatikan hasil PRINT.

Jika inputnya:

```
SET @NamaPelanggan = 'Budi';
```

maka SQL yang terbentuk kira-kira:

```
SELECT *
FROM Pelanggan
WHERE NamaPelanggan = 'Budi'
```

B. Uji input berbahaya

Sekarang ubah:

```
SET @NamaPelanggan = 'Budi';
```

Menjadi:

```
SET @NamaPelanggan = ''' OR ''1''='''1''';
```

Kemudian:

```
DECLARE @NamaPelanggan VARCHAR(100);
DECLARE @SQL VARCHAR(MAX);

SET @NamaPelanggan = '' OR '1'='1';

SET @SQL =
'SELECT *
FROM Pelanggan
WHERE NamaPelanggan = '' + @NamaPelanggan + ''';

PRINT @SQL;
```

70 %

Messages

```
SELECT *
FROM Pelanggan
WHERE NamaPelanggan = '' OR '1'='1'
```

Completion time: 2026-08-24T12:40:22.3020096+07:00

kita akan melihat bahwa **input pengguna ikut membentuk struktur SQL**.

Kesimpulan yang harus ditemukan mahasiswa

Masalahnya bukan pada SELECT-nya, tetapi pada cara aplikasi **menggabungkan input pengguna langsung ke dalam SQL**.

PRAKTIKUM 17 — Parameterized Query

Sekarang mahasiswa membuat versi yang aman.

```
DECLARE @NamaPelanggan VARCHAR(100);

SET @NamaPelanggan = 'Budi';

SELECT *
FROM Pelanggan
WHERE NamaPelanggan = @NamaPelanggan;
```

Kemudian uji:

```
SET @NamaPelanggan = '' OR '1'='1';
```

Jalankan kembali.

Hasil

SQL Server mencari nama pelanggan yang nilainya sama dengan:

' OR '1'='1'

bukan menjalankan input tersebut sebagai SQL.